

LLM Agent Assignment and Token Optimization

Take-Home Assignment

Background

This is a take-home assignment for assessing the match between your skills and knowledge the skills needed for a specific MEng project. The MEng project is ideally suitable for a student who has (1) experience with python, (2) familiarity with mathematical modeling and optimization, (3) skills for handling csv and json files, (4) academic writing, and (5) familiarity with AI and LLM terminology and concepts.

Based on the reports received, the candidate with the best report will be offered an opportunity to take an 11-month MEng project (3 semester) that starts in Fall 2026. There is no funding unfortunately. The candidate will receive supervision through regular meetings, and a grade in early August 2027 based on a pre-defined marking scheme and the deliverables of the project completed before August 2027.

Optimization Problem Setting

Consider a system that receives a set of queries and has access to several LLM agents. The agents have different capabilities, costs, and inference speeds. The number of output tokens allocated to an agent can also affect the quality and cost of its answer.

For each query i , agent j , and allowed token budget k , the provided dataset contains:

- **Observed answer quality:** q_{ijk}
- **Token budget:** t_{ijk}
- **Monetary cost:** c_{ijk}
- **Expected model latency:** l_{ijk}
- **Query characteristics:** basic characteristics of the query

The optimization task is to decide which agent should answer each query and how many tokens should be allocated to it.

For any set of N queries, use the following default constraints:

- **Total token budget:** $B_T = 1000N$
- **Total monetary budget:** $B_C = \$0.00022N$
- **Agent capacities:** A: 30% N , B: 25% N , C: 25% N , D: 25% N , E: 20% N

Round agent capacities down to the nearest integer where needed. You may also study tighter or looser budgets.

Tasks

1. **Formulate the Optimization Problem.** Define a mathematical optimization model for the problem. Clearly specify the decision variables, objective function, constraints, and assumptions. Explain what type of mathematical optimization problem you obtain and any transformations you use.
2. **Implement and Analyze the Optimization Model.** Implement your formulation in Python using an optimization solver of your choice. Use the train queries in `data.csv`. Study how the assignments and objective value change under different resource budgets; at minimum, evaluate the default budget, a tighter budget (both budgets multiplied by 0.7), and a looser budget (both multiplied by 1.5).
3. **Design a Fast Assignment Method.** Solving an optimization model from scratch may be too slow for a production LLM system, where routing decisions may need to be made in milliseconds. Design a method that produces high-quality agent/token assignments with very low decision-time overhead. The choice and design are part of the assignment. Use the train queries for development and evaluate the final method on the test queries. Use the optimization solution as a reference when available. The method should plausibly be usable for real-time routing and should not simply call the full optimization model for each decision.
4. **Open-Ended Extension.** Choose one meaningful extension below, or propose a different extension of your own. You do not need to fully implement it. Define the problem, propose a concrete design or method, and explain how you would evaluate it.
 - **Uncertainty-aware / robust assignment:** quality, cost, latency, or token requirements may be uncertain or misestimated.
 - **Adaptive / sequential routing:** observe intermediate results and then stop, allocate more tokens, escalate, or request verification.
 - **Self-improving / online routing:** learn from previous assignments and outcomes so routing improves during deployment.
 - **Multi-agent workflow design:** allow agents to cooperate, verify each other, or execute dependent/parallel tasks.
 - **Resource-aware real-time routing:** account for changing queue length, capacity, availability, latency, or workload.

This part is intentionally open-ended. We are interested in how you turn the direction into a well-defined problem, what assumptions you make, what method you propose, and how you would determine whether it works.

Data

`data.csv` contains one row for every available query-agent-token alternative, measured from real LLM API calls. The columns are:

query_id, split, query_type, input_tokens, agent_id, token_budget, observed_quality, cost_usd, latency_ms

- **Queries:** 375 queries (266 train, 109 test) of five types: math_olympiad, math_comp, coding, knowledge, multihop_qa.
- **Agents:** Five agents A–E are five different small models. Three of them reason (hidden chain-of-thought) before answering; two are instruction-only.
- **token_budget:** the max_tokens cap given to the model (512, 1024, 2048, 4096). It caps hidden reasoning and answer together.
- **observed_quality:** the graded correctness of the single answer the model produced (0/1 for math, code and multiple choice; token-F1 in [0, 1] for multihop_qa). It is an observed outcome, not an expectation. For Tasks 1–2, treat the observed quality values as the utility coefficients available to the offline optimizer.
- **cost_usd:** computed from the tokens actually billed (input plus generated, including hidden reasoning) at the provider's list prices, so it is not necessarily monotone in token_budget.
- **latency_ms:** the measured wall-clock time of the model call under concurrent load. It is noisy.
- **input_tokens:** the prompt length reported by the API.
- **queries.jsonl:** contains the prompt text of every query, for methods that want to use the query text.

Do not use the test split when fitting or tuning a learning-based method.

For Task 3, observed_quality for test queries is provided only for evaluation and must not be used as an input to the routing method or otherwise inspected when making routing decisions.

Submission Requirements

Please email s.aref@utoronto.ca the following by the deadline that was given to you:

1. **GitHub Repository.** Provide a link to a public GitHub repository containing your source code, experiments, and instructions required to reproduce the results.
2. **Summary Report.** Submit a PDF report of at most 1000 words. Your report is expected to include equations and references, and may include table/figure as needed. Put the GitHub repository link on the first line of the report. Include the formulation, main computational results, fast assignment method, and proposed extension. Please use the filename `Firstname_Lastname_summary.pdf`.
3. **CV and transcript(s).** Please include in the email, your up-to-date academic CV and the unofficial English transcripts(s) from your previous programs. Please use the filename `Firstname_Lastname_CV.pdf` and use the filename `Firstname_Lastname_transcript.pdf`.

Hints

- If you have any questions about this assignment, feel free to consult online resources and references or AI tools. Please do not send questions by email.
- Using AI tools is fine, but you must declare what tools you have used and make efforts to check and correct anything you receive from an AI tool. You are ultimately responsible for the correctness of the written work you submit, and having a complete understanding of all the technical details for the work you submit.
- Fix random seeds and report solver/library versions and computational settings.
- Python is recommended, but you may use any suitable solver or ML framework.
- If any part of this assignment is not fully defined in the above instructions, take it as an opportunity to make your own choices and justify them in your summary.
- There is no single correct solution for Tasks 3 and 4.